Extra EXERCISE of React Js

AN ISO 9001:2015 CERTIFIED INSTITUTE

COSMOS

The Real Training Centre

Extra Exercise of React Js

Function Component

1. Task Manager App

Description: A simple task manager application that allows users to add, remove, and mark tasks as complete.

```
import React, { useState } from 'react';
// Task Component
const Task = ({ task, onRemove, onToggleComplete }) => (
  <div style={{ textDecoration: task.completed ? 'line-
through' : 'none' }}>
    <span>{task.name}</span>
    <button onClick={() =>
onToggleComplete(task.id)}>Complete</button>
    <button onClick={() =>
onRemove(task.id)}>Delete</button>
  </div>
);
// TaskManager Component
const TaskManager = () => {
  const [tasks, setTasks] = useState([]);
  const [taskName, setTaskName] = useState('');
  const addTask = () => {
    if (taskName.trim() === '') return;
    setTasks([...tasks, { id: Date.now(), name: taskName,
completed: false }]);
    setTaskName('');
  };
  const removeTask = (id) => {
    setTasks(tasks.filter(task => task.id !== id));
  };
  const toggleComplete = (id) => {
    setTasks(tasks.map(task =>
      task.id === id ? { ...task, completed: !task.completed
} : task
    ));
  };
  return (
    <div>
      <input
        type="text"
```

```
          value={taskName}
          onChange={(e) => setTaskName(e.target.value)}
          placeholder="New task"
        />
        <button onClick={addTask}>Add Task</button>
        <div>
          {tasks.map(task => (
            <Task
              key={task.id}
              task={task}
              onRemove={removeTask}
              onToggleComplete={toggleComplete}
            />
          ))}
        </div>
      </div>
    );
};
export default TaskManager;
```

2. Color Picker

Description: A color picker that allows users to select a color from a palette and displays the color with its hex value.

```
import React, { useState } from 'react';
// ColorPicker Component
const ColorPicker = () => {
  const [color, setColor] = useState('#000000');

  const handleColorChange = (event) => {
    setColor(event.target.value);
  };
  return (
    <div>
      <input
        type="color"
        value={color}
        onChange={handleColorChange}
      />
      <p>Selected Color: {color}</p>
    </div>
  );
};
export default ColorPicker;
```

3. Counter with Multiple Buttons

Description: A counter app with multiple buttons to increase, decrease, reset, and double the counter value.

```
import React, { useState } from 'react';
// Counter Component
const Counter = () => {
  const [count, setCount] = useState(0);
  const increase = () => setCount(count + 1);
  const decrease = () => setCount(count - 1);
  const reset = () => setCount(0);
  const double = () => setCount(count * 2);
  return (
    <div>
      <h1>{count}</h1>
      <button onClick={increase}>Increase</button>
      <button onClick={decrease}>Decrease</button>
      <button onClick={reset}>Reset</button>
      <button onClick={double}>Double</button>
    </div>
  );
};
export default Counter;
```

4. Simple Calculator

Description: A basic calculator that performs addition, subtraction, multiplication, and division.

```
import React, { useState } from 'react';
// Calculator Component
const Calculator = () => {
  const [result, setResult] = useState(0);
  const [input, setInput] = useState('');
  const handleInput = (value) => {
    setInput(input + value);
  };
  const clearInput = () => {
    setInput('');
  };
  const calculateResult = () => {
    try {
      setResult(eval(input));  // Use eval to calculate the
expression
      setInput('');
    } catch (e) {
```

```
        setResult('Error');
      }
    };
    return (
      <div>
        <div>
          <input type="text" value={input} readOnly />
        </div>
        <div>
          <button onClick={() => handleInput('1')}>1</button>
          <button onClick={() => handleInput('2')}>2</button>
          <button onClick={() => handleInput('3')}>3</button>
          <button onClick={() => handleInput('+')}>+</button>
        </div>
        <div>
          <button onClick={() => handleInput('4')}>4</button>
          <button onClick={() => handleInput('5')}>5</button>
          <button onClick={() => handleInput('6')}>6</button>
          <button onClick={() => handleInput('-')}>-</button>
        </div>
        <div>
          <button onClick={() => handleInput('7')}>7</button>
          <button onClick={() => handleInput('8')}>8</button>
          <button onClick={() => handleInput('9')}>9</button>
          <button onClick={() => handleInput('*')}>*</button>
        </div>
        <div>
          <button onClick={() => handleInput('0')}>0</button>
          <button onClick={clearInput}>C</button>
          <button onClick={calculateResult}>=</button>
          <button onClick={() => handleInput('/')}>/</button>
        </div>
        <div>
          <h2>Result: {result}</h2>
        </div>
      </div>
    );
  };
  export default Calculator;
```

5. Quote Generator

Description: A quote generator that displays a random inspirational quote when a button is clicked.

```
import React, { useState } from 'react';
```

```
// Quote Generator Component
const QuoteGenerator = () => {
  const quotes = [
    "The only way to do great work is to love what you do. - Steve Jobs",
    "Success is not final, failure is not fatal: It is the courage to continue that counts. - Winston Churchill",
    "It is never too late to be what you might have been. - George Eliot",
    "Act as if what you do makes a difference. - William James",
    "Success usually comes to those who are too busy to be looking for it. - Henry David Thoreau"
  ];
  const [quote, setQuote] = useState('');
  const generateQuote = () => {
    const randomIndex = Math.floor(Math.random() * quotes.length);
    setQuote(quotes[randomIndex]);
  };
  return (
    <div>
      <h1>Inspirational Quote</h1>
      <p>{quote || "Click the button to generate a quote."}</p>
      <button onClick={generateQuote}>Generate Quote</button>
    </div>
  );
};
export default QuoteGenerator;
```

Class Component

6. Todo List App

Description: A simple Todo list app that allows users to add tasks, mark them as complete, and delete them.

```
import React, { Component } from 'react';
// Todo Component
class Todo extends Component {
  render() {
    const { task, onDelete, onToggleComplete } = this.props;
    return (
```

```
      <div style={{ textDecoration: task.completed ? 'line-
through' : 'none' }}>
        <span>{task.name}</span>
        <button onClick={() =>
onToggleComplete(task.id)}>Complete</button>
        <button onClick={() =>
onDelete(task.id)}>Delete</button>
      </div>
    );
  }
}
// TodoList Component
class TodoList extends Component {
  constructor(props) {
    super(props);
    this.state = {
      tasks: [],
      taskName: '',
    };
  }
  addTask = () => {
    if (this.state.taskName.trim() === '') return;
    const newTask = {
      id: Date.now(),
      name: this.state.taskName,
      completed: false,
    };
    this.setState({
      tasks: [...this.state.tasks, newTask],
      taskName: '',
    });
  };
  removeTask = (id) => {
    this.setState({
      tasks: this.state.tasks.filter((task) => task.id !==
id),
    });
  };

  toggleComplete = (id) => {
    this.setState({
      tasks: this.state.tasks.map((task) =>
        task.id === id ? { ...task, completed:
!task.completed } : task
```

```
      ),
    });
  };
  handleChange = (e) => {
    this.setState({
      taskName: e.target.value,
    });
  };
  render() {
    return (
      <div>
        <input
          type="text"
          value={this.state.taskName}
          onChange={this.handleChange}
          placeholder="New task"
        />
        <button onClick={this.addTask}>Add Task</button>
        <div>
          {this.state.tasks.map((task) => (
            <Todo
              key={task.id}
              task={task}
              onDelete={this.removeTask}
              onToggleComplete={this.toggleComplete}
            />
          ))}
        </div>
      </div>
    );
  }
}
export default TodoList;
```

7. Color Picker

Description: A color picker that allows the user to choose a color and display its hexadecimal value.

```
import React, { Component } from 'react';
class ColorPicker extends Component {
  constructor(props) {
    super(props);
    this.state = {
      color: '#000000',
    };
```

```
  }
  handleColorChange = (event) => {
    this.setState({
      color: event.target.value,
    });
  };

  render() {
    return (
      <div>
        <input
          type="color"
          value={this.state.color}
          onChange={this.handleColorChange}
        />
        <p>Selected Color: {this.state.color}</p>
      </div>
    );
  }
}
export default ColorPicker;
```

8. Counter App with Multiple Actions

Description: A counter app that allows users to increase, decrease, reset, and double the counter value.

```
import React, { Component } from 'react';
class Counter extends Component {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
    };
  }
  increase = () => {
    this.setState({ count: this.state.count + 1 });
  };
  decrease = () => {
    this.setState({ count: this.state.count - 1 });
  };
  reset = () => {
    this.setState({ count: 0 });
  };
  double = () => {
    this.setState({ count: this.state.count * 2 });
```

```
  };
  render() {
    return (
      <div>
        <h1>{this.state.count}</h1>
        <button onClick={this.increase}>Increase</button>
        <button onClick={this.decrease}>Decrease</button>
        <button onClick={this.reset}>Reset</button>
        <button onClick={this.double}>Double</button>
      </div>
    );
  }
}
export default Counter;
```

9. Simple Calculator

Description: A basic calculator app to perform arithmetic operations like addition, subtraction, multiplication, and division.

```
import React, { Component } from 'react';
class Calculator extends Component {
  constructor(props) {
    super(props);
    this.state = {
      input: '',
      result: 0,
    };
  }
  handleInput = (value) => {
    this.setState({ input: this.state.input + value });
  };
  clearInput = () => {
    this.setState({ input: '' });
  };
  calculateResult = () => {
    try {
      this.setState({ result: eval(this.state.input), input:
'' });
    } catch (e) {
      this.setState({ result: 'Error', input: '' });
    }
  };
  render() {
    return (
      <div>
```

```
        <input type="text" value={this.state.input} readOnly
/>
        <div>
          <button onClick={() =>
this.handleInput('1')}>1</button>
          <button onClick={() =>
this.handleInput('2')}>2</button>
          <button onClick={() =>
this.handleInput('3')}>3</button>
          <button onClick={() =>
this.handleInput('+')}>+</button>
        </div>
        <div>
          <button onClick={() =>
this.handleInput('4')}>4</button>
          <button onClick={() =>
this.handleInput('5')}>5</button>
          <button onClick={() =>
this.handleInput('6')}>6</button>
          <button onClick={() => this.handleInput('-')}>-
</button>
        </div>
        <div>
          <button onClick={() =>
this.handleInput('7')}>7</button>
          <button onClick={() =>
this.handleInput('8')}>8</button>
          <button onClick={() =>
this.handleInput('9')}>9</button>
          <button onClick={() =>
this.handleInput('*')}>*</button>
        </div>
        <div>
          <button onClick={() =>
this.handleInput('0')}>0</button>
          <button onClick={this.clearInput}>C</button>
          <button onClick={this.calculateResult}>=</button>
          <button onClick={() =>
this.handleInput('/')}>/</button>
        </div>
        <h2>Result: {this.state.result}</h2>
      </div>
    );
  }}export default Calculator;
```

High Order Component

10. Random Quote Generator

Description: A random quote generator that displays a new quote every time the user clicks a button.

```
import React, { Component } from 'react';
class QuoteGenerator extends Component {
  constructor(props) {
    super(props);
    this.state = {
      quote: '',
    };
    this.quotes = [
      'The only way to do great work is to love what you do.
- Steve Jobs',
      'Success is not final, failure is not fatal: It is the
courage to continue that counts. - Winston Churchill',
      'It is never too late to be what you might have been.
- George Eliot',
      'Act as if what you do makes a difference. - William
James',
      'Success usually comes to those who are too busy to be
looking for it. - Henry David Thoreau',
    ];
  }
  generateQuote = () => {
    const randomIndex = Math.floor(Math.random() *
this.quotes.length);
    this.setState({ quote: this.quotes[randomIndex] });
  };
  render() {
    return (
      <div>
        <h1>Inspirational Quote</h1>
        <p>{this.state.quote || 'Click the button to
generate a quote.'}</p>
        <button onClick={this.generateQuote}>Generate
Quote</button>
      </div>
    );
  }
}export default QuoteGenerator;
```

11. WithLoadingSpinner (HOC for Loading State)

Description: A Higher-Order Component that adds a loading spinner to any component, displaying a loading state until the data is fetched or a task is completed.

```
import React, { Component } from 'react';
// Higher-Order Component for loading spinner
const withLoadingSpinner = (WrappedComponent) => {
  return class extends Component {
    render() {
      const { isLoading, ...otherProps } = this.props;
      return isLoading ? (
        <div>Loading...</div>
      ) : (
        <WrappedComponent {...otherProps} />
      );
    }
  };
};
// Main Component
class DataDisplay extends Component {
  render() {
    return (
      <div>
        <h1>Data is loaded!</h1>
        <p>{this.props.data}</p>
      </div>
    );
  }
}
const DataDisplayWithLoading =
withLoadingSpinner(DataDisplay);

class App extends Component {
  state = {
    isLoading: true,
    data: null,
  };
  componentDidMount() {
    setTimeout(() => {
      this.setState({
        isLoading: false,
        data: 'Fetched data from the API!',
      });
    }, 2000); // Simulate data fetch delay
  }
```

```
    render() {
      return (
        <div>
          <DataDisplayWithLoading
            isLoading={this.state.isLoading}
            data={this.state.data}
          />
        </div>
      );
    }
}export default App;
```

12. WithCounter (HOC for Counter Logic)

Description: A Higher-Order Component that provides counter functionality (increment/decrement) to any component.

```
import React, { Component } from 'react';
// Higher-Order Component for counter functionality
const withCounter = (WrappedComponent) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = { count: 0 };
    }
    increment = () => {
      this.setState((prevState) => ({ count:
prevState.count + 1 }));
    };
    decrement = () => {
      this.setState((prevState) => ({ count:
prevState.count - 1 }));
    };
    render() {
      return (
        <WrappedComponent
          count={this.state.count}
          increment={this.increment}
          decrement={this.decrement}
          {...this.props}
        />
      );
    }
  };
};
// Display Component that uses the counter logic
```

```
class CounterDisplay extends Component {
  render() {
    return (
      <div>
        <h1>Counter: {this.props.count}</h1>
        <button
onClick={this.props.increment}>Increase</button>
        <button
onClick={this.props.decrement}>Decrease</button>
      </div>
    );
  }
}
const EnhancedCounterDisplay =
withCounter(CounterDisplay);
class App extends Component {
  render() {
    return (
      <div>
        <EnhancedCounterDisplay />
      </div>
    );
  }
}
export default App;
```

13. **WithPermission** (HOC for Permission Check)

Description: A Higher-Order Component that checks if the user has the necessary permissions to access a component. If not, it shows an access denied message.

```
import React, { Component } from 'react';
// Higher-Order Component for permission checking
const withPermission = (WrappedComponent,
requiredPermission) => {
  return class extends Component {
    render() {
      const { userPermission, ...otherProps } =
this.props;
      if (userPermission !== requiredPermission) {
        return <div>Access Denied. You do not have the
required permission.</div>;
      }
      return <WrappedComponent {...otherProps} />;
    }
```

```
  };
};
// Component that requires permission
class AdminPanel extends Component {
  render() {
    return <h1>Welcome to the Admin Panel</h1>;
  }
}
const EnhancedAdminPanel = withPermission(AdminPanel,
'admin');
class App extends Component {
  render() {
    return (
      <div>
        <EnhancedAdminPanel userPermission="user" />
        <EnhancedAdminPanel userPermission="admin" />
      </div>
    );
  }
}
export default App;
```

14. WithErrorBoundary (HOC for Error Handling)

Description: A Higher-Order Component that catches JavaScript errors in a component tree and displays an error fallback UI.

```
import React, { Component } from 'react';
// Higher-Order Component for error boundary
const withErrorBoundary = (WrappedComponent) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = { hasError: false };
    }
    static getDerivedStateFromError() {
      return { hasError: true };
    }
   componentDidCatch(error, info) {
      console.log(error, info);
    }
    render() {
      if (this.state.hasError) {
        return <div>Something went wrong. Please try again
later.</div>;
```

```
      }
      return <WrappedComponent {...this.props} />;
    }
  };
};
// A component that will throw an error
class BuggyComponent extends Component {
  render() {
    if (this.props.hasError) {
      throw new Error('Something went wrong!');
    }
    return <h1>No error, all good!</h1>;
  }
}
const BuggyComponentWithErrorBoundary =
withErrorBoundary(BuggyComponent);
class App extends Component {
  render() {
    return (
      <div>
        <BuggyComponentWithErrorBoundary hasError={false}
/>
        <BuggyComponentWithErrorBoundary hasError={true}
/>
      </div>
    );
  }
}
export default App;
```

15. WithWindowWidth (HOC for Window Size)

Description: A Higher-Order Component that provides the current window width to any component, allowing you to react to window resizing events.

```
import React, { Component } from 'react';
// Higher-Order Component for window width
const withWindowWidth = (WrappedComponent) => {
  return class extends Component {
    constructor(props) {
      super(props);
      this.state = {
        windowWidth: window.innerWidth,
      };
    }
    updateWindowWidth = () => {
```

```
      this.setState({ windowWidth: window.innerWidth });
    };
    componentDidMount() {
      window.addEventListener('resize',
this.updateWindowWidth);
    }
    componentWillUnmount() {
      window.removeEventListener('resize',
this.updateWindowWidth);
    }
    render() {
      return <WrappedComponent
windowWidth={this.state.windowWidth} {...this.props} />;
    }
  };
};
// Display window width
class WindowWidthDisplay extends Component {
  render() {
    return <h1>Window Width:
{this.props.windowWidth}px</h1>;
  }
}
const EnhancedWindowWidthDisplay =
withWindowWidth(WindowWidthDisplay);
class App extends Component {
  render() {
    return (
      <div>
        <EnhancedWindowWidthDisplay />
      </div>
    );
  }
}
export default App;
```

Pure Component

16. Simple Counter (Pure Component)

Description: A counter that increments or decrements the value. This program demonstrates the usage of a pure component, as the Counter component will only re-render when the state changes.

```
import React, { PureComponent } from 'react';
class Counter extends PureComponent {
  constructor(props) {
    super(props);
    this.state = {
      count: 0,
    };
  }
  increment = () => {
    this.setState({ count: this.state.count + 1 });
  };
  decrement = () => {
    this.setState({ count: this.state.count - 1 });
  };
  render() {
    console.log('Counter rendered');
    return (
      <div>
        <h1>{this.state.count}</h1>
        <button onClick={this.increment}>Increase</button>
        <button onClick={this.decrement}>Decrease</button>
      </div>
    );
  }
}
export default Counter;
```

17. User Profile (Pure Component)

Description: A component that displays a user's profile with their name and age. The profile only re-renders when the props (name or age) change.

```
import React, { PureComponent } from 'react';
class UserProfile extends PureComponent {
  render() {
    console.log('UserProfile rendered');
    return (
      <div>
```

```
          <h2>Name: {this.props.name}</h2>
          <p>Age: {this.props.age}</p>
        </div>
      );
    }
  }
  class App extends React.Component {
    state = {
      user: { name: 'John Doe', age: 30 },
    };
    changeAge = () => {
      this.setState({ user: { ...this.state.user, age:
  this.state.user.age + 1 } });
    };

    render() {
      return (
        <div>
          <UserProfile name={this.state.user.name}
  age={this.state.user.age} />
          <button onClick={this.changeAge}>Increase
  Age</button>
        </div>
      );
    }
  }
  export default App;
```

18. Memoized List (Pure Component)

Description: A list of items that updates when the list data changes. The list component only re-renders if the list items change, thanks to PureComponent.

```
import React, { PureComponent } from 'react';
class List extends PureComponent {
  render() {
    console.log('List rendered');
    return (
      <ul>
        {this.props.items.map((item, index) => (
          <li key={index}>{item}</li>
        ))}
      </ul>
    );
  }
}
```

```
class App extends React.Component {
  state = {
    items: ['Apple', 'Banana', 'Orange'],
  };
 addItem = () => {
    this.setState({ items: [...this.state.items, 'New
Item'] });
  };
  render() {
    return (
      <div>
        <List items={this.state.items} />
        <button onClick={this.addItem}>Add Item</button>
      </div>
    );
  }
}
export default App;
```

19. Product Card (Pure Component)

Description: A product card component that only re-renders when the product's name, price, or availability changes.

```
import React, { PureComponent } from 'react';
class ProductCard extends PureComponent {
  render() {
    console.log('ProductCard rendered');
    const { name, price, available } = this.props;
    return (
      <div>
        <h3>{name}</h3>
        <p>Price: ${price}</p>
        <p>{available ? 'In Stock' : 'Out of Stock'}</p>
      </div>
    );
  }
}
class App extends React.Component {
  state = {
    product: { name: 'Laptop', price: 999, available: true
},
  };
  toggleAvailability = () => {
    this.setState((prevState) => ({
      product: { ...prevState.product, available:
!prevState.product.available },
```

```
    }));
  };
  render() {
    return (
      <div>
        <ProductCard {...this.state.product} />
        <button onClick={this.toggleAvailability}>Toggle
Availability</button>
      </div>
    );
  }
}
export default App;
```

20. Todo Item (Pure Component)

Description: A todo item component that only re-renders when its status (completed or not) changes.

```
import React, { PureComponent } from 'react';
class TodoItem extends PureComponent {
  render() {
    const { todo, toggleCompletion } = this.props;
    return (
      <div>
        <input
          type="checkbox"
          checked={todo.completed}
          onChange={() => toggleCompletion(todo.id)}
        />
        <span style={{ textDecoration: todo.completed ?
'line-through' : 'none' }}>
          {todo.text}
        </span>
      </div>
    );
  }
}
class App extends React.Component {
  state = {
    todos: [
      { id: 1, text: 'Learn React', completed: false },
      { id: 2, text: 'Build an App', completed: false },
    ],
  };
  toggleCompletion = (id) => {
```

```
      this.setState({
        todos: this.state.todos.map((todo) =>
          todo.id === id ? { ...todo, completed:
!todo.completed } : todo
        ),
      });
  };
  render() {
    return (
      <div>
        {this.state.todos.map((todo) => (
          <TodoItem key={todo.id} todo={todo}
toggleCompletion={this.toggleCompletion} />
        ))}
      </div>
    );
  }
}
export default App;
```

Control Component

21. Controlled Input Field (Text Input)

Description: A simple controlled input field where the value of the input is tied to the component's state, and any change to the input field updates the state.

```
import React, { Component } from 'react';
class ControlledInput extends Component {
  constructor(props) {
    super(props);
    this.state = {
      inputValue: '',
    };
  }
  handleChange = (e) => {
    this.setState({ inputValue: e.target.value });
  };
  render() {
    return (
      <div>
        <input
          type="text"
          value={this.state.inputValue}
          onChange={this.handleChange}
        />
```

```
          <p>You typed: {this.state.inputValue}</p>
        </div>
      );
    }
}
export default ControlledInput;
```

22. Controlled Checkbox (Accept Terms)

Description: A checkbox that tracks whether the user accepts the terms and conditions, using controlled state.

```
import React, { Component } from 'react';
class TermsCheckbox extends Component {
  constructor(props) {
    super(props);
    this.state = {
      isChecked: false,
    };
  }
  handleCheckboxChange = () => {
    this.setState({ isChecked: !this.state.isChecked });
  };
  render() {
    return (
      <div>
        <label>
          <input
            type="checkbox"
            checked={this.state.isChecked}
            onChange={this.handleCheckboxChange}
          />
          I accept the terms and conditions
        </label>
        <p>{this.state.isChecked ? 'You accepted the
terms.' : 'You have not accepted the terms.'}</p>
      </div>
    );
  }
}
export default TermsCheckbox;
```

23. Controlled Select Dropdown (Choose Color)

Description: A dropdown menu where users can select their favorite color, and the selected color is managed by the component's state.

```
import React, { Component } from 'react';
class ColorPicker extends Component {
  constructor(props) {
    super(props);
    this.state = {
      selectedColor: 'red',
    };
  }
  handleSelectChange = (e) => {
    this.setState({ selectedColor: e.target.value });
  };
  render() {
    return (
      <div>
        <select value={this.state.selectedColor}
onChange={this.handleSelectChange}>
          <option value="red">Red</option>
          <option value="green">Green</option>
          <option value="blue">Blue</option>
        </select>
        <p>Your selected color is:
{this.state.selectedColor}</p>
      </div>
    );
  }
}
export default ColorPicker;
```

24. Controlled Form with Multiple Inputs (User Info)

Description: A form where users enter their name and email, and the form state is controlled by the parent component.

```
import React, { Component } from 'react';
class UserInfoForm extends Component {
  constructor(props) {
    super(props);
    this.state = {
      name: '',
      email: '',
    };
  }
```

```
  handleChange = (e) => {
    this.setState({ [e.target.name]: e.target.value });
  };
  handleSubmit = (e) => {
    e.preventDefault();
    console.log('User Info:', this.state);
  };
  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>Name:</label>
          <input
            type="text"
            name="name"
            value={this.state.name}
            onChange={this.handleChange}
          />
        </div>
        <div>
          <label>Email:</label>
          <input
            type="email"
            name="email"
            value={this.state.email}
            onChange={this.handleChange}
          />
        </div>
        <button type="submit">Submit</button>
      </form>
    );
  }
}
export default UserInfoForm;
```

25. Controlled Radio Buttons (Gender Selection)

Description: A set of radio buttons that allow users to select their gender. The selection is managed by the component's state.

```
import React, { Component } from 'react';
class GenderSelection extends Component {
  constructor(props) {
    super(props);
    this.state = {
      gender: 'male',
```

```
    };
  }
  handleGenderChange = (e) => {
    this.setState({ gender: e.target.value });
  };
  render() {
    return (
      <div>
        <label>
          <input
            type="radio"
            name="gender"
            value="male"
            checked={this.state.gender === 'male'}
            onChange={this.handleGenderChange}
          />
          Male
        </label>
        <label>
          <input
            type="radio"
            name="gender"
            value="female"
            checked={this.state.gender === 'female'}
            onChange={this.handleGenderChange}
          />
          Female
        </label>
        <p>Selected Gender: {this.state.gender}</p>
      </div>
    );
  }
}
export default GenderSelection;
```

Uncontrol Component

26. Simple Uncontrolled Input (Text Input)

Description: A basic uncontrolled input field where the input value is accessed directly from the DOM using a ref.

```
import React, { Component } from 'react';
class UncontrolledInput extends Component {
  constructor(props) {
    super(props);
    this.inputRef = React.createRef(); // Create a ref for
the input element
  }
  handleSubmit = (e) => {
    e.preventDefault();
    alert('Submitted: ' + this.inputRef.current.value); //
Access value via ref
  };
  render() {
    return (
      <div>
        <form onSubmit={this.handleSubmit}>
          <input type="text" ref={this.inputRef} />
          <button type="submit">Submit</button>
        </form>
      </div>
    );
  }
}
export default UncontrolledInput;
```

27. Uncontrolled Checkbox (Accept Terms)

Description: A checkbox to accept terms and conditions, with the checkbox's state handled by the DOM rather than React's state.

```
import React, { Component } from 'react';
class TermsCheckbox extends Component {
  constructor(props) {
    super(props);
    this.checkboxRef = React.createRef(); // Create a ref
for the checkbox
  }
  handleSubmit = (e) => {
```

```
    e.preventDefault();
    alert(
      'Accepted terms: ' +
        (this.checkboxRef.current.checked ? 'Yes' : 'No')
// Access checked status
    );
  };
  render() {
    return (
      <div>
        <form onSubmit={this.handleSubmit}>
          <label>
            <input
              type="checkbox"
              ref={this.checkboxRef}
            />
            I accept the terms and conditions
          </label>
          <button type="submit">Submit</button>
        </form>
      </div>
    );
  }
}
export default TermsCheckbox;
```

28. Uncontrolled Form with Multiple Inputs (User Info)

Description: A form with multiple inputs (name, email) where each input value is accessed using refs.

```
import React, { Component } from 'react';
class UserInfoForm extends Component {
  constructor(props) {
    super(props);
    this.nameRef = React.createRef(); // Ref for name
input
    this.emailRef = React.createRef(); // Ref for email
input
  }
  handleSubmit = (e) => {
    e.preventDefault();
    alert(
      `Name: ${this.nameRef.current.value}, Email:
${this.emailRef.current.value}`
    );
```

```
  };
  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>Name:</label>
          <input type="text" ref={this.nameRef} />
        </div>
        <div>
          <label>Email:</label>
          <input type="email" ref={this.emailRef} />
        </div>
        <button type="submit">Submit</button>
      </form>
    );
  }
}
export default UserInfoForm;
```

29. Uncontrolled Textarea (Message Submission)

Description: A textarea for entering a message where the message is handled by the DOM rather than React's state.

```
import React, { Component } from 'react';
class MessageForm extends Component {
  constructor(props) {
    super(props);
    this.textareaRef = React.createRef(); // Ref for the
textarea element
  }

  handleSubmit = (e) => {
    e.preventDefault();
    alert('Message Submitted: ' +
this.textareaRef.current.value); // Access textarea value
  };
  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>Message:</label>
          <textarea ref={this.textareaRef}></textarea>
        </div>
        <button type="submit">Submit</button>
      </form>
```

```
      );
    }
  }
  export default MessageForm;
```

30. Uncontrolled Radio Buttons (Gender Selection)

Description: A set of radio buttons where the selected value is managed by the DOM, and the selected gender is accessed on form submission.

```
import React, { Component } from 'react';
class GenderSelection extends Component {
  constructor(props) {
    super(props);
    this.genderRef = React.createRef(); // Ref for the
radio buttons
  }
  handleSubmit = (e) => {
    e.preventDefault();
    alert('Selected Gender: ' +
this.genderRef.current.value); // Access selected radio
button
  };
  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>
            <input
              type="radio"
              name="gender"
              value="male"
              ref={this.genderRef}
            />
            Male
          </label>
          <label>
            <input
              type="radio"
              name="gender"
              value="female"
              ref={this.genderRef}
            />
            Female
          </label>
        </div>
```

```
          <button type="submit">Submit</button>
        </form>
      );
    }
}
export default GenderSelection;
```

Use Reference

31. Auto-Focus Input (Input Focus on Load)

Description: A simple program that uses a ref to automatically focus on an input field when the component is loaded.

```
import React, { Component } from 'react';
class AutoFocusInput extends Component {
  constructor(props) {
    super(props);
    this.inputRef = React.createRef(); // Create a ref for
the input element
  }
  componentDidMount() {
    this.inputRef.current.focus(); // Focus the input
field after the component mounts
  }
  render() {
    return (
      <div>
        <input type="text" ref={this.inputRef}
placeholder="Focus on me!" />
      </div>
    );
  }
}
export default AutoFocusInput;
```

32. Video Player Control (Play and Pause)

Description: A simple video player that uses a ref to control the play and pause of a video element.

```
import React, { Component } from 'react';
class VideoPlayer extends Component {
  constructor(props) {
    super(props);
```

```
    this.videoRef = React.createRef(); // Create a ref for
the video element
  }
  playVideo = () => {
    this.videoRef.current.play(); // Play the video using
the ref
  };
  pauseVideo = () => {
    this.videoRef.current.pause(); // Pause the video
using the ref
  };
  render() {
    return (
      <div>
        <video ref={this.videoRef} width="400" controls>
          <source
src="https://www.w3schools.com/html/mov_bbb.mp4"
type="video/mp4" />
          Your browser does not support the video tag.
        </video>
        <div>
          <button onClick={this.playVideo}>Play</button>
          <button onClick={this.pauseVideo}>Pause</button>
        </div>
      </div>
    );
  }
}
export default VideoPlayer;
```

33. Form Submission (Accessing Form Elements)

Description: A form with uncontrolled input elements, where the form data is accessed using refs upon submission.

```
import React, { Component } from 'react';
class ContactForm extends Component {
  constructor(props) {
    super(props);
    this.nameRef = React.createRef(); // Create a ref for
the name input
    this.emailRef = React.createRef(); // Create a ref for
the email input
  }
  handleSubmit = (e) => {
    e.preventDefault();
```

```
    alert(
      `Name: ${this.nameRef.current.value}, Email:
${this.emailRef.current.value}`
    ); // Access input values using refs
  };
  render() {
    return (
      <form onSubmit={this.handleSubmit}>
        <div>
          <label>Name:</label>
          <input type="text" ref={this.nameRef} />
        </div>
        <div>
          <label>Email:</label>
          <input type="email" ref={this.emailRef} />
        </div>
        <button type="submit">Submit</button>
      </form>
    );
  }
}
export default ContactForm;
```

34. Animating a Div (Scroll to a Specific Element)

Description: A program that scrolls to a specific div when the "Go to Section" button is clicked using a ref.

```
import React, { Component } from 'react';
class ScrollToSection extends Component {
  constructor(props) {
    super(props);
    this.sectionRef = React.createRef(); // Create a ref
for the section div
  }
  scrollToSection = () => {
    this.sectionRef.current.scrollIntoView({ behavior:
'smooth' }); // Scroll to the div
  };

  render() {
    return (
      <div>
        <button onClick={this.scrollToSection}>Go to
Section</button>
```

```
        <div style={{ height: '1000px', backgroundColor:
'#f0f0f0' }}>
          Scroll down to see the section
        </div>
        <div
          ref={this.sectionRef}
          style={{ height: '200px', backgroundColor:
'#a0d0f0' }}
        >
          This is the section to scroll to!
        </div>
      </div>
    );
  }
}
export default ScrollToSection;
```

35. Controlling a Modal (Show and Hide)

Description: A modal component that can be shown or hidden by controlling its visibility with a ref.

```
import React, { Component } from 'react';
class Modal extends Component {
  constructor(props) {
    super(props);
    this.modalRef = React.createRef(); // Create a ref for
the modal div
  }
  openModal = () => {
    this.modalRef.current.style.display = 'block'; // Show
the modal
  };
  closeModal = () => {
    this.modalRef.current.style.display = 'none'; // Hide
the modal
  };
  render() {
    return (
      <div>
        <button onClick={this.openModal}>Open
Modal</button>
        <div
          ref={this.modalRef}
          style={{
            display: 'none',
```

```
              position: 'fixed',
              top: '50%',
              left: '50%',
              transform: 'translate(-50%, -50%)',
              backgroundColor: 'white',
              padding: '20px',
              boxShadow: '0 4px 8px rgba(0,0,0,0.2)',
            }}
          >
            <h2>Modal Content</h2>
            <button onClick={this.closeModal}>Close</button>
          </div>
        </div>
      );
    }
  }
  export default Modal;
```

Use Effect

36. Timer with useEffect (Count Down Timer)

Description: A countdown timer that decreases every second. useEffect is used to set up the interval, and the timer stops when the component is unmounted.

```
import React, { useState, useEffect } from 'react';
const CountdownTimer = () => {
  const [time, setTime] = useState(10); // Initial time
set to 10 seconds
  useEffect(() => {
    // Set up interval to decrease time every second
    const interval = setInterval(() => {
      setTime((prevTime) => {
        if (prevTime === 1) clearInterval(interval); //
Stop timer at 0
        return prevTime - 1;
      });
    }, 1000);

    // Cleanup interval when component unmounts or time
reaches 0
    return () => clearInterval(interval);
  }, []); // Empty dependency array ensures this effect
runs only once when the component mounts
  return (
```

```
      <div>
        <h1>Countdown: {time} seconds</h1>
      </div>
    );
};
export default CountdownTimer;
```

37. Fetching Data with useEffect (API Request)

Description: This example shows how useEffect can be used to fetch data from an API when the component mounts and display the fetched data.

```
import React, { useState, useEffect } from 'react';
const FetchData = () => {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    // Fetch data from an API
    fetch('https://jsonplaceholder.typicode.com/posts/1')
      .then((response) => response.json())
      .then((json) => {
        setData(json);
        setLoading(false); // Set loading to false when
data is fetched
      })
      .catch((error) => {
        console.error('Error fetching data:', error);
        setLoading(false);
      });
  }, []); // Empty dependency array ensures the effect
runs only once
  return (
    <div>
      {loading ? (
        <p>Loading...</p>
      ) : (
        <div>
          <h2>{data.title}</h2>
          <p>{data.body}</p>
        </div>
      )}
    </div>
  );};export default FetchData;
```

38. Window Resize Event with useEffect

Description: This program listens for window resizing events and updates the state with the new window width. useEffect is used to attach and clean up the event listener.

```
import React, { useState, useEffect } from 'react';
const WindowResize = () => {
  const [width, setWidth] = useState(window.innerWidth);
  useEffect(() => {
    // Function to update window width
    const handleResize = () =>
setWidth(window.innerWidth);

    // Add event listener for resize event
    window.addEventListener('resize', handleResize);

    // Cleanup event listener when component unmounts
    return () => window.removeEventListener('resize',
handleResize);
  }, []); // Empty dependency array ensures the effect
runs only once when the component mounts
  return (
    <div>
      <h1>Window Width: {width}px</h1>
    </div>
  );
};
export default WindowResize;
```

39. Subscription Example with useEffect

Description: A mock subscription component that subscribes to a service (simulated using setInterval) and cleans up when the component unmounts.

```
import React, { useState, useEffect } from 'react';
const Subscription = () => {
  const [messages, setMessages] = useState([]);
  useEffect(() => {
    // Simulate receiving messages from a subscription
service
    const interval = setInterval(() => {
      setMessages((prevMessages) => [
        ...prevMessages,
        `New message at ${new
Date().toLocaleTimeString()}`,
      ]);
    }, 3000); // Simulate receiving a message every 3
seconds
```

```
    // Cleanup: unsubscribe when the component unmounts
    return () => clearInterval(interval);
  }, []); // Empty dependency array ensures this effect
runs only once
  return (
    <div>
      <h1>Messages:</h1>
      <ul>
        {messages.map((msg, index) => (
          <li key={index}>{msg}</li>
        ))}
      </ul>
    </div>
  );
};
export default Subscription;
```

40. Updating Document Title with useEffect.

Description: This example demonstrates how useEffect can be used to update the document's title based on the state of the component.

```
import React, { useState, useEffect } from 'react';
const DocumentTitleUpdater = () => {
  const [count, setCount] = useState(0);

  useEffect(() => {
    // Update the document title with the current count
    document.title = `Count: ${count}`;
  }, [count]); // Dependency array ensures the effect runs
only when 'count' changes
  return (
    <div>
      <h1>Count: {count}</h1>
      <button onClick={() => setCount(count +
1)}>Increment</button>
    </div>
  );
};
export default DocumentTitleUpdater;
```